Formal Verification Report: Aquarius Fees Collector

- Competition: Aquarius Audit + Certora Formal Verification
- Repository: https://github.com/alexzoid-eth/aquarius-cantina-fv
- Scope: fees_collector, access_control
- Date: June 2025
- Author: @alexzoid
- Certora Sunbeam (Soroban) Prover version: 7.26.0

About Aquarius Protocol

Aquarius is a decentralized protocol governed by DAO voting with AQUA tokens, built on the Stellar blockchain. It provides automated market maker (AMM) functionality through Soroban smart contracts, enabling liquidity provision, token swaps, and fee collection across multiple pool types including stableswap and standard liquidity pools. The protocol features role-based access control with time-delayed administrative operations and emergency mode capabilities.

About the Fees Collector Module

The fees collector module manages protocol fee collection and distribution with a comprehensive role-based access control system. It implements a multi-role administration model with six distinct roles (Admin, Emergency Admin, Rewards Admin, Operations Admin, Pause Admin, and Emergency Pause Admin). Administrative changes to critical roles (Admin and Emergency Admin) require a two-step, time-delayed transfer process: commit followed by apply after a mandatory delay period. The module also supports upgradeable contracts with similar time-delayed commit-apply patterns, and includes an emergency mode that allows immediate upgrades when activated by the Emergency Admin.

Competition Scope

For this formal verification competition, the following crates are in scope:

- `fees_collector/src/contract.rs` - Main contract entry points for fee collection and administration
- `fees_collector/src/interface.rs` - Contract interface definitions
- `fees_collector/src/storage.rs` - Storage helpers
- `access_control/src/emergency.rs` - Emergency mode management
- `access_control/src/management.rs` - Role management operations
- `access_control/src/storage.rs` - Storage key mappings for roles
- `access_control/src/transfer.rs` - Ownership transfer with time delays
- `upgrade/src/lib.rs` - Contract upgrade logic

Table of Contents

- Formal Verification Methodology
  - Types of Properties
    - Parametric Rules
    - Rules
  - Verification Process
    - Setup
    - Crafting Properties
  - Assumptions
    - Safe Assumptions
    - Unsafe Assumptions
- Verification Properties
  - Sanity
  - State Transitions
  - Permissions
  - Integrity
  - High Level
- Manual Mutations Testing
  - Access Control - Emergency
  - Access Control - Management
  - Access Control - Storage
  - Access Control - Transfer
  - Fees Collector - Contract
  - Upgrade - Library
- Violated Properties
  - Issue #1 - Upgrade Deadline Lifecycle
  - Issue #2 - Admin Deadline Clear
- Setup and Execution Instructions
  - Certora Prover Installation
  - Verification Execution

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. It complements techniques like testing and fuzzing, which can only sometimes detect bugs based on predefined properties. In contrast, Certora FV examines all possible states and execution paths in a contract.

Simply put, the formal verification process involves crafting properties (similar to writing tests) in native RUST language and submitting them alongside compiled programs to a remote prover. This prover essentially transforms the program bytecode and rules into a mathematical model and determines the validity of rules.

Types of Properties

When constructing properties in formal verification, we mainly deal with two types: **Parametric Rules** and **Rules**.

Parametric Rules

- Properties verified across ALL external functions automatically. Implemented with the parametric infrastructure that generates one property check per external function.
- Process:
    1. Initialize ghost storage from rule parameters.
    2. Assume realistic timestamp.
    3. Log all storage variables.
    4. Execute external function.
    5. Log all storage variables again.
    6. Assert the property holds.
- Example: "Admin roles can only change through the time-delayed transfer process."
- Use Case: Ensures **State Transition** and **Permission** properties - critical invariants that MUST never be violated across all functions.

Rules

- Flexible checks for specific behaviors or conditions.
- Structure:
    1. Setup: Initialize contract state (e.g., "set admin role").
    2. Execution: Simulate contract behavior by calling external functions.
    3. Verification:
        - Use `cvlr_assert!()` to check if a condition is **always true** (e.g., "emergency mode toggle works correctly").
        - Use `cvlr_satisfy!()` to verify a condition is **reachable** (e.g., "immediate upgrade is reachable in emergency mode").
- Example: "A committed upgrade can be reverted, clearing the deadline."
- Use Case: Verifies **Integrity** and **High Level** properties, from simple getter/setter consistency to complex multi-step workflows.

Verification Process

The process is divided into two stages: **Setup** and **Crafting Properties**.

Setup

This stage prepares the contract and prover for verification. Use conditional source code compilation with `#[cfg(feature = "certora")]`.

- Mirror storage read/write operations into ghost state variables via the `ghost_state` crate to reduce complexity.
- Create harness functions in `access_control_harness.rs` and `upgrade_harness.rs` to expose internal library functions for verification.
- Prove parametric properties as a foundation for mutation testing.

Crafting Properties

This stage defines and implements the properties:

- Write properties in **plain English** for clarity.
- Categorize properties by purpose. Parametric style (`sanity.rs`, `state_transition.rs`, `permissions.rs`) properties are verified across all external functions. Non-parametric (`integrity.rs`, `high_level.rs`) as regular rules targeting specific behaviors.
- Use ghost state mirroring for efficient storage tracking across function calls.

Assumptions

Assumptions simplify the verification process and are classified as **Safe** or **Unsafe**. Safe assumptions are backed by valid logic or required by the environment. Unsafe assumptions are made to reduce complexity, potentially limiting coverage.

Safe Assumptions

- Block timestamps are always non-zero (`e.ledger().timestamp() > 0`)
- Ghost state is initialized from actual persistent storage before each verification

Unsafe Assumptions

Loop Unrolling

- Vector iterations limited to 2 iterations (`loop_iter = 2` in configs)

Optimistic Loop

- Optimistic loop handling enabled (`optimistic_loop = true` in configs)

Verification Properties

The verification properties are categorized into the following types:

1. **Sanity (SA)**: Basic reachability of function execution paths

2. **State Transitions (ST)**: Time-delayed transitions for roles and upgrades
3. **Permissions (PER)**: Role-based access control for state changes
4. **Integrity (INT)**: Getter/setter consistency and storage key mappings
5. **High Level (HL)**: Multi-step workflows and emergency mode behaviors

Each job status linked to a corresponding run in the dashboard with a specific status:

- ✅ completed successfully
- ⚠️ reached global timeout
- ❌ violated

Sanity

Basic checks ensuring contract functions remain accessible and operational.

All sanity properties are stored in sanity.rs and ⚠️ passed verification (with `sanity_apply_transfer_ownership` reaching global timeout).

Source	Rule	Description	Caught mutations
SA-01	sanity	All external functions remain callable	-

State Transitions

These properties verify that state changes occur correctly during contract operations, ensuring time-delayed transitions for critical role and upgrade operations.

All state transition properties are stored in state_transition.rs and passed verification across multiple runs (links per property below).

Source	Rule	Description	Prover	Caught mutations
ST-01	state_transition_admin_role	Admin role can only change through time-delayed transfer process	✅ ✅	❌management_0
ST-02	state_transition_emergency_admin_role	Emergency admin role can only change through time-delayed transfer process	✅ ✅	❌management_0, ❌transfer_4
ST-03	state_transition_admin_init_once	Initial admin can only be set once	✅ ✅	❌management_0
ST-04	state_transition_admin_never_zero	Roles can never be removed once set	✅	-
ST-05	state_transition_admin_deadline_lifecycle	Admin deadline transitions: 0→non-zero (commit) and non-zero→0 (apply/revert)	✅ ✅ ✅	❌transfer_5
ST-06	state_transition_admin_deadline_clear	Future admin cleared when deadline goes to 0	❌ Issue #2	-
ST-07	state_transition_em_admin_deadline_lifecycle	Emergency admin deadline lifecycle transitions	✅ ✅	-

Source	Rule	Description	Prover	Caught mutations
ST-08	state_transition_em_admin_deadline_clear	Future emergency admin cleared when deadline goes to 0	❌ Issue #2	-
ST-09	state_transition_future_admin_consistency	If deadline doesn't change, future admin must remain same	✅	❌transfer_5
ST-10	state_transition_future_em_admin_consistency	If deadline doesn't change, future emergency admin must remain same	✅	❌transfer_5
ST-11	state_transition_emergency_mode_behavior	Emergency mode allows immediate upgrades but not immediate transfers	✅	-
ST-12	state_transition_upgrade_deadline_lifecycle	Upgrade deadline transitions: 0 → non-zero (commit) and non-zero → 0 (apply/revert)	✅❌ Issue #1	-
ST-13	state_transition_future_wasm_consistency	If deadline doesn't change, future wasm must remain same	✅	-

Permissions

These properties verify that only authorized roles can modify specific state variables.

All permission properties are stored in permissions.rs and passed verification across multiple runs.

Source	Rule	Description	Prover	Caught mutations
PER-01	permissions_admin_role_changes	Only current admin can change admin role	✅	-
PER-02	permissions_other_admins_role_changes	Only admin can change other admin roles	✅	-
PER-03	permissions_admin_transfer_deadline_changes	Only admin can change admin transfer deadlines	✅✅	❌contract_3, ❌contract_5
PER-04	permissions_em_admin_transfer_deadline_changes	Only admin can change emergency admin transfer deadlines	⚠️	❌contract_3, ❌contract_6
PER-05	permissions_future_admin_changes	Only admin can change future admin; consistent with deadline transitions	✅✅	❌contract_5
PER-06	permissions_future_em_admin_changes	Only admin can change future emergency admin; consistent with deadline transitions	✅✅	❌contract_5
PER-07	permissions_emergency_mode_changes	Only emergency admin can change emergency mode	✅	-

Source	Rule	Description	Prover	Caught mutations
PER-08	permissions_upgrade_deadline_changes	Only admin can change upgrade deadlines	✅	❌contract_0, ❌contract_3
PER-09	permissions_future_wasm_changes	Only admin can change future wasm; consistent with deadline transitions	✅	❌contract_0

Integrity

Properties ensuring data consistency and correctness throughout operations.

All integrity properties are stored in integrity.rs and passed verification: ✅ main, ✅ address_has_role, ✅ role_address_safe, ✅ role_address_single successfully.

Emergency Mode Integrity

Source	Rule	Description	Caught mutations
INT-01	integrity_emergency_mode	Setting emergency mode correctly stores the value	❌emergency_0, ❌contract_4, ❌contract_2

Admin and Role Integrity

Source	Rule	Description	Caught mutations
INT-02	integrity_init_admin	Initial admin address is correctly stored	-
INT-03	integrity_future_address	Future address set by commit_transfer_ownership is stored correctly	❌storage_2, ❌transfer_1, ❌transfer_5
INT-04	integrity_commit_transfer_deadline	Commit sets deadline to timestamp + ADMIN_ACTIONS_DELAY	❌storage_2, ❌transfer_1, ❌transfer_5
INT-05	integrity_role_address_single	Single role address set and retrieved correctly	❌management_1
INT-06	integrity_role_addresses_multiple	Multiple role addresses (vector) set and retrieved correctly	-
INT-07	integrity_role_address_safe	Safe role address getter returns correct value	❌management_1
INT-08	integrity_init_all_roles	All roles initialized correctly	❌storage_0
INT-09	integrity_privileged_addresses	Privileged addresses set and retrieved correctly	❌management_1, ❌storage_0
INT-10	integrity_address_has_role	address_has_role returns true when address has role	❌management_1
INT-11	integrity_assert_address_has_role	assert_address_has_role doesn't panic when address has role	-
INT-12	integrity_assert_address_has_role_negative	assert_address_has_role panics when address doesn't have role	❌management_1

Upgrade Integrity

Source	Rule	Description	Caught mutations
INT-13	integrity_commit_upgrade_future_wasm	Commit upgrade correctly stores future wasm hash	❌contract_7
INT-14	integrity_commit_upgrade_deadline	Commit upgrade sets deadline to timestamp + ADMIN_ACTIONS_DELAY	❌contract_7, ❌lib_1, ❌lib_0
INT-15	integrity_upgrade_deadline_direct	Upgrade deadline put and get are consistent	-
INT-16	integrity_future_wasm_direct	Future wasm put and get are consistent	-

Storage Key Mapping Integrity

Source	Rule	Description	Caught mutations
INT-17	integrity_get_key_admin	Storage key mapping for Admin role (1 → 1)	-
INT-18	integrity_get_key_emergency_admin	Storage key mapping for EmergencyAdmin role (2 → 2)	-
INT-19	integrity_get_key_rewards_admin	Storage key mapping for RewardsAdmin role (3 → 3)	❌storage_0
INT-20	integrity_get_key_operations_admin	Storage key mapping for OperationsAdmin role (4 → 4)	-
INT-21	integrity_get_key_pause_admin	Storage key mapping for PauseAdmin role (5 → 5)	-
INT-22	integrity_get_key_emergency_pause_admin	Storage key mapping for EmergencyPauseAdmin role (6 → 6)	-
INT-23	integrity_get_future_key_admin	Future storage key mapping for Admin role (1 → 7)	-
INT-24	integrity_get_future_key_emergency_admin	Future storage key mapping for EmergencyAdmin role (2 → 8)	❌storage_1
INT-25	integrity_get_future_deadline_key_admin	Future deadline key mapping for Admin role (1 → 9)	❌storage_2
INT-26	integrity_get_future_deadline_key_emergency_admin	Future deadline key mapping for EmergencyAdmin role (2 → 10)	-

Role Requirement Integrity

Source	Rule	Description	Caught mutations
INT-27	integrity_require_rewards_admin_with_rewards_admin	require_rewards_admin_or_owner accepts rewards_admin	-
INT-28	integrity_require_rewards_admin_with_admin	require_rewards_admin_or_owner accepts admin	-
INT-29	integrity_require_rewards_admin_negative	require_rewards_admin_or_owner rejects unauthorized	❌management_1

Source	Rule	Description	Caught mutations
INT-30	integrity_require_operations_admin_with_operations_admin	require_operations_admin_owner accepts operations_admin	-
INT-31	integrity_require_operations_admin_with_admin	require_operations_admin_owner accepts admin	-
INT-32	integrity_require_operations_admin_negative	require_operations_admin_owner rejects unauthorized	❌management_1
INT-33	integrity_require_pause_emergency_with_pause_admin	require_pause_emergency_admin accepts pause_admin	-
INT-34	integrity_require_pause_emergency_with_emergency_pause_admin	require_pause_emergency_admin accepts emergency_pause_admin	-
INT-35	integrity_require_pause_emergency_with_admin	require_pause_emergency_admin accepts admin	-
INT-36	integrity_require_pause_admin_with_pause_admin	require_pause_admin_or_owner accepts pause_admin	-
INT-37	integrity_require_pause_admin_with_admin	require_pause_admin_or_owner accepts admin	-
INT-38	integrity_require_pause_admin_negative	require_pause_admin_or_owner rejects unauthorized	❌management_1

High Level

Complex business logic and protocol-specific properties.

All high-level properties are stored in high_level.rs and ✅ passed verification successfully.

Source	Rule	Description	Caught mutations
HL-01	high_level_revert_ownership_transfer	Ownership transfer can be reverted before deadline, leaving admin unchanged	-
HL-02	high_level_upgrade_immediate_in_emergency	Upgrades can be applied immediately only in emergency mode	❌contract_7
HL-03	high_level_upgrade_reachable	Reachability: immediate upgrade path is reachable in emergency mode	❌management_1
HL-04	high_level_revert_upgrade	Upgrade can be reverted, clearing the deadline	❌contract_8
HL-05	high_level_revert_upgrade_reachable	Reachability: upgrade revert path is reachable	❌management_1, ❌contract_8
HL-06	high_level_emergency_mode_toggle	Emergency mode can be toggled on and off by emergency admin	❌emergency_0, ❌contract_4, ❌contract_2
HL-07	high_level_concurrent_transfers_blocked	Concurrent ownership transfers are blocked through mutual exclusion	-
HL-08	high_level_deadline_expiry_prevents_apply	Applying ownership transfer before deadline panics	-

Manual Mutations Testing

This section documents the manual mutations applied to the fees collector contract and its dependencies. Each mutation introduces a single-line code defect to test the specifications' ability to detect bugs. 24 mutations total: 1 emergency + 2 management + 3 storage + 6 transfer + 9 contract + 3 upgrade library.

Mutation coverage summary:

- **Caught**: 19/24 (79%)
- **Not caught**: 5/24 (transfer_0, transfer_2, transfer_3, contract_1, lib_2)

Access Control - Emergency

mutations/access_control/emergency/emergency_0.rs

Skips setting/clearing emergency mode in `set_emergency_mode()`. Vulnerability: system cannot enter emergency mode, perpetuating any risk by breaking the mitigation mechanism.

```
// e.storage().instance().set(&DataKey::EmergencyMode, value); MUTANT
```

Caught by:

- ❌ **HL-06**: high_level_emergency_mode_toggle - Emergency mode can be toggled on and off
- ❌ **INT-01**: integrity_emergency_mode - Setting emergency mode correctly stores the value

Access Control - Management

mutations/access_control/management/management_0.rs

Negated test of `is_transfer_delayed()` in `set_role_address()`. Vulnerability: allows immediate change of critical roles that should require a time delay.

```
if addr.is_some() && !role.is_transfer_delayed() {  // MUTANT (negation)
```

Caught by:

- ❌ **ST-01**: state_transition_admin_role - Admin role can only change through time-delayed transfer
- ❌ **ST-02**: state_transition_emergency_admin_role - Emergency admin role requires time-delayed transfer
- ❌ **ST-03**: state_transition_admin_init_once - Initial admin can only be set once

mutations/access_control/management/management_1.rs

Skips setting role in `set_role_addresses()`. Vulnerability: role addresses can never actually be stored.

```
// self.0.storage().instance().set(&key, address); MUTANT
```

Caught by:

- ❌ **HL-03**: high_level_upgrade_reachable - Immediate upgrade path is reachable in emergency mode

- ❌ **HL-05**: high_level_revert_upgrade_reachable - Upgrade revert path is reachable
- ❌ **INT-10**: integrity_address_has_role - address_has_role returns true when address has role
- ❌ **INT-12**: integrity_assert_address_has_role_negative - assert_address_has_role panics when address doesn't have role
- ❌ **INT-09**: integrity_privileged_addresses - Privileged addresses set and retrieved correctly
- ❌ **INT-05**: integrity_role_address_single - Single role address set and retrieved correctly
- ❌ **INT-07**: integrity_role_address_safe - Safe role address getter returns correct value
- ❌ **INT-29**: integrity_require_rewards_admin_negative - require_rewards_admin_or_owner rejects unauthorized
- ❌ **INT-32**: integrity_require_operations_admin_negative - require_operations_admin_owner rejects unauthorized
- ❌ **INT-38**: integrity_require_pause_admin_negative - require_pause_admin_or_owner rejects unauthorized

Access Control - Storage

mutations/access_control/storage/storage_0.rs

In `get_key()`, gives wrong key (OperationsAdmin rather than Operator) for the RewardsAdmin role. Vulnerability: extra permissions granted to RewardsAdmin and OperationsAdmin due to address storage collision.

```
    Role::RewardsAdmin => DataKey::OperationsAdmin, // MUTANT: changed from
DataKey::Operator
```

Caught by:

- ❌ **INT-19**: integrity_get_key_rewards_admin - Storage key mapping for RewardsAdmin role
- ❌ **INT-08**: integrity_init_all_roles - All roles initialized correctly
- ❌ **INT-09**: integrity_privileged_addresses - Privileged addresses set and retrieved correctly

mutations/access_control/storage/storage_1.rs

In `get_future_key()`, gives wrong key (FutureAdmin rather than FutureEmergencyAdmin) for the EmergencyAdmin role. Vulnerability: full Admin permissions may be granted to EmergencyAdmin role due to address storage collision.

```
    Role::EmergencyAdmin => DataKey::FutureAdmin, // MUTANT: changed from
DataKey::FutureEmergencyAdmin
```

Caught by:

- ❌ **INT-24**: integrity_get_future_key_emergency_admin - Future storage key mapping for EmergencyAdmin role

mutations/access_control/storage/storage_2.rs

In `get_future_deadline_key()`, gives wrong key (FutureAdmin rather than TransferOwnershipDeadline) for the Admin role. Vulnerability: Admin role transfer cannot be initiated, will revert with type error when checking existing deadline.

```
    Role::Admin => DataKey::FutureAdmin, // MUTANT: changed from
DataKey::TransferOwnershipDeadline
```

Caught by:

- ❌ **INT-25**: integrity_get_future_deadline_key_admin - Future deadline key mapping for Admin role
- ❌ **INT-03**: integrity_future_address - Future address set by commit_transfer_ownership
- ❌ **INT-04**: integrity_commit_transfer_deadline - Commit sets deadline correctly

Access Control - Transfer

mutations/access_control/transfer/transfer_0.rs

Skips ownership transfer in `apply_transfer_ownership()`. Vulnerability: roles cannot be changed, violating design principle.

```
    // storage.set(&self.get_key(role), &future_address); MUTANT
```

Not caught by any rule.

mutations/access_control/transfer/transfer_1.rs

Ignores stored deadline in `get_transfer_ownership_deadline()` and returns 0. Vulnerability: `apply_transfer_ownership()` always reverts so roles cannot be changed.

```
    let value = 0; // MUTANT: self.0.storage().instance().get(&key).unwrap_or(0);
```

Caught by:

- ❌ **INT-04**: integrity_commit_transfer_deadline - Commit sets deadline correctly
- ❌ **INT-03**: integrity_future_address - Future address stored correctly

mutations/access_control/transfer/transfer_2.rs

Ignores requested deadline in `put_transfer_ownership_deadline()` and stores 0. Vulnerability: `apply_transfer_ownership()` always reverts so roles cannot be changed.

```
    self.0.storage().instance().set(&key, &0); // MUTANT: replaced value by 0
```

Not caught by any rule (only timeouts).

mutations/access_control/transfer/transfer_3.rs

Fails to reset deadline in `revert_transfer_ownership()`. Vulnerability: transfer of Admin and EmergencyAdmin cannot be canceled.

```
    //self.put_transfer_ownership_deadline(role, 0); // MUTANT
```

Not caught by any rule.

mutations/access_control/transfer/transfer_4.rs

Skips reset of deadline in `apply_transfer_ownership()`. Vulnerability: first transfer of Admin or EmergencyAdmin role blocks all future transfers of that role.

```
    // self.put_transfer_ownership_deadline(role, 0); MUTANT
```

Caught by:

- ❌ **ST-02**: state_transition_emergency_admin_role - Emergency admin role requires time-delayed transfer

mutations/access_control/transfer/transfer_5.rs

`commit_transfer_ownership()` always sets a deadline in the past. Vulnerability: role changes that should be delayed can be applied immediately after being initiated.

```
    let deadline = self.0.ledger().timestamp() - ADMIN_ACTIONS_DELAY;  // MUTANT (+ to -)
```

Caught by:

- ❌ **ST-05**: state_transition_admin_deadline_lifecycle - Admin deadline transitions
- ❌ **ST-09**: state_transition_future_admin_consistency - Future admin value consistency
- ❌ **ST-10**: state_transition_future_em_admin_consistency - Future emergency admin value consistency
- ❌ **INT-04**: integrity_commit_transfer_deadline - Commit sets deadline correctly
- ❌ **INT-03**: integrity_future_address - Future address stored correctly

Fees Collector - Contract

mutations/fees_collector/contract/contract_0.rs (public)

Removed authorization check in `commit_upgrade()` restricting to Admin role. Vulnerability: anyone can initiate a software update with arbitrary code.

```
    // AccessControl::new(&e).assert_address_has_role(&admin, &Role::Admin); MUTANT
```

Caught by:

- ❌ **PER-08**: permissions_upgrade_deadline_changes - Only admin can change upgrade deadlines
- ❌ **PER-09**: permissions_future_wasm_changes - Only admin can change future wasm

mutations/fees_collector/contract/contract_1.rs (public)

Removed authorization check in `apply_transfer_ownership()` restricting to Admin role. Vulnerability: anyone can complete the transfer of Admin and EmergencyAdmin roles.

```
// access_control.assert_address_has_role(&admin, &Role::Admin); MUTANT
```

Not caught by any rule (only timeout in `sanity_apply_transfer_ownership`).

mutations/fees_collector/contract/contract_2.rs (public)

`get_emergency_mode()` always returns false. Vulnerability: `fees_collector.get_emergency_mode()` unreliable.

```
false // MUTANT: always returns false, changed from `get_emergency_mode(&e)`
```

Caught by:

- ❌ **HL-06**: high_level_emergency_mode_toggle - Emergency mode can be toggled
- ❌ **INT-01**: integrity_emergency_mode - Setting emergency mode correctly stores the value

mutations/fees_collector/contract/contract_3.rs

Removed authorization check in `revert_upgrade()` restricting to Admin role. Vulnerability: anyone can cancel a software update.

```
// AccessControl::new(&e).assert_address_has_role(&admin, &Role::Admin); MUTANT
```

Caught by:

- ❌ **PER-03**: permissions_admin_transfer_deadline_changes - Only admin can change admin transfer deadlines
- ❌ **PER-08**: permissions_upgrade_deadline_changes - Only admin can change upgrade deadlines
- ❌ **PER-04**: permissions_em_admin_transfer_deadline_changes - Only admin can change emergency admin transfer deadlines

mutations/fees_collector/contract/contract_4.rs

Skips setting/clearing emergency mode in `set_emergency_mode()`. Vulnerability: system cannot enter emergency mode, perpetuating any risk by breaking the mitigation mechanism.

```
// set_emergency_mode(&e, &value); MUTANT
```

Caught by:

- ❌ **HL-06**: high_level_emergency_mode_toggle - Emergency mode can be toggled
- ❌ **INT-01**: integrity_emergency_mode - Setting emergency mode correctly stores the value

mutations/fees_collector/contract/contract_5.rs

Disables `revert_transfer_ownership()`. Vulnerability: transfer of Admin and EmergencyAdmin cannot be canceled.

```
// access_control.revert_transfer_ownership(&role); MUTANT
```

Caught by:

- ❌ **PER-03**: permissions_admin_transfer_deadline_changes - Only admin can change admin transfer deadlines
- ❌ **PER-05**: permissions_future_admin_changes - Only admin can change future admin
- ❌ **PER-06**: permissions_future_em_admin_changes - Only admin can change future emergency admin

mutations/fees_collector/contract/contract_6.rs

Removed authorization check in `commit_transfer_ownership()` restricting to Admin role. Vulnerability: anyone can initiate transfer of Admin and EmergencyAdmin roles to an arbitrary address.

```
// access_control.assert_address_has_role(&admin, &Role::Admin); MUTANT
```

Caught by:

- ❌ **PER-04**: permissions_em_admin_transfer_deadline_changes - Only admin can change emergency admin transfer deadlines

mutations/fees_collector/contract/contract_7.rs

Fails to store code hash in `commit_upgrade()`. Vulnerability: upgrades can never be completed.

```
// commit_upgrade(&e, &new_wasm_hash); MUTANT
```

Caught by:

- ❌ **HL-02**: high_level_upgrade_immediate_in_emergency - Upgrades can be applied immediately only in emergency mode
- ❌ **INT-13**: integrity_commit_upgrade_future_wasm - Commit upgrade correctly stores future wasm hash
- ❌ **INT-14**: integrity_commit_upgrade_deadline - Commit upgrade sets deadline correctly

mutations/fees_collector/contract/contract_8.rs

Disables `revert_upgrade()`. Vulnerability: upgrades can never be canceled.

```
// revert_upgrade(&e); MUTANT
```

Caught by:

- ❌ **HL-04**: high_level_revert_upgrade - Upgrade can be reverted, clearing the deadline
- ❌ **HL-05**: high_level_revert_upgrade_reachable - Upgrade revert path is reachable

Upgrade - Library

mutations/upgrade/lib/lib_0.rs

Upgrades always have deadline 0 in `commit_upgrade()`. Vulnerability: upgrades can only be completed in emergency mode.

```
    let deadline = 0; // MUTANT: e.ledger().timestamp() + UPGRADE_DELAY;
```

Caught by:

- ❌ **INT-14**: integrity_commit_upgrade_deadline - Commit upgrade sets deadline correctly

mutations/upgrade/lib/lib_1.rs

Skips setting deadline in `commit_upgrade()`. Vulnerability: upgrades can only be completed in emergency mode.

```
    // put_upgrade_deadline(e, &deadline); MUTANT
```

Caught by:

- ❌ **INT-14**: integrity_commit_upgrade_deadline - Commit upgrade sets deadline correctly

mutations/upgrade/lib/lib_2.rs

Skips reset of deadline in `apply_upgrade()`. Vulnerability: any past upgrade can be applied again.

```
    // put_upgrade_deadline(e, &0); MUTANT
```

Not caught by any rule (only timeout in `sanity_apply_transfer_ownership`).

Violated Properties

These properties were intentionally violated during verification, revealing actual issues in the contract code.

Issue #1 - Upgrade Deadline Lifecycle

GitHub Issue: alexzoid-eth/aquarius-cantina-fv#1

Finding: The `state_transition_upgrade_deadline_lifecycle` property is violated for `apply_upgrade` and `revert_upgrade` functions.

Affected Rules (from fees_collector_state_transition_upgrade_deadlines_violated.conf):

- `state_transition_upgrade_deadline_lifecycle_apply_upgrade`
- `state_transition_upgrade_deadline_lifecycle_revert_upgrade`

Description: The upgrade deadline lifecycle check in `check_upgrade_deadline_lifecycle()` validates that when a deadline transitions from non-zero to zero (apply or revert), the timestamp must have passed the deadline in normal mode. The violation indicates that the `apply_upgrade` and `revert_upgrade` functions do not correctly enforce the upgrade deadline lifecycle constraints.

```
fn check_upgrade_deadline_lifecycle(
    e: &Env,
    deadline_before: u64,
    deadline_after: u64,
    _future_before: Option<impl Clone>,
    future_after: Option<impl Clone>,
    emergency_mode_before: bool,
    _emergency_mode_after: bool,
) -> bool {
    // ...
    } else if deadline_before != 0 && deadline_after == 0 {
        // Transition from non-zero to 0 (apply or revert)
        if emergency_mode_before {
            future_after.is_none()
        } else {
            e.ledger().timestamp() >= deadline_before && future_after.is_none()
        }
    }
    // ...
}
```

Issue #2 - Admin Deadline Clear

GitHub Issue: [alexzoid-eth/aquarius-cantina-fv#2]

Finding: The `state_transition_admin_deadline_clear` and `state_transition_em_admin_deadline_clear` properties are violated for `apply_transfer_ownership` and `revert_transfer_ownership` functions.

Affected Rules (from [fees_collector_state_transition_admin_transfer_deadlines_violated.conf]):

- `state_transition_admin_deadline_clear_apply_transfer_ownership`
- `state_transition_admin_deadline_clear_revert_transfer_ownership`
- `state_transition_em_admin_deadline_clear_apply_transfer_ownership`
- `state_transition_em_admin_deadline_clear_revert_transfer_ownership`

Description: When the admin transfer deadline transitions from non-zero to zero (during apply or revert), the `future_admin` (or `future_em_admin`) should be cleared to `None`. The violation shows that the contract does not properly clear the future address when completing or reverting an ownership transfer. This was noted in the `check_admin_deadline_lifecycle()` function where the relevant check was commented out due to this issue.

```
fn check_admin_deadline_lifecycle(
    e: &Env,
    deadline_before: u64,
    deadline_after: u64,
    future_after: Option<impl Clone>,
```

```
) -> bool {
    if deadline_before == 0 && deadline_after != 0 {
        deadline_after == e.ledger().timestamp() + ADMIN_ACTIONS_DELAY &&
future_after.is_some()
    // } else if deadline_before != 0 && deadline_after == 0 {
        // Transition from non-zero to 0 (apply or revert)
        // @note This executable path is violated due
        //  to https://github.com/alexzoid-eth/aquarius-cantina-fv/issues/2
        // future_after.is_none()
    } else if deadline_before != 0 && deadline_after != 0 {
        deadline_before == deadline_after
    } else {
        true
    }
}
```

Setup and Execution Instructions

Certora Prover Installation

For step-by-step installation steps refer to the Certora documentation.

Verification Execution

1. Build the fees_collector contract with Certora features:

```
cd fees_collector
just build
```

2. Run a specific verification:

```
cd fees_collector/confs
certoraSorobanProver <config_file>.conf
```

3. Run all verifications:

```
cd fees_collector/confs
./run_all.sh
```

4. Run mutation testing for a specific category:

```
cd fees_collector/confs
./run_all_mutations.sh
```